

Experiment No.7

- **Title:-** Implement the CYK (Cocke–Younger–Kasami) parsing algorithm (or Chart Parsing technique).
- **Aim:-** To Implement the CYK (Cocke–Younger–Kasami) parsing algorithm (or Chart Parsing technique).

- **Objectives:-**

1. Understand Context-Free Grammars (CFG) and Chomsky Normal Form (CNF).
2. Implement the CYK parsing algorithm in Python.
3. Test the parser with example grammars and sentences.
4. (Optional) Understand differences between CYK and Chart Parsing and implement a simple chart parser if required.
5. Analyze ambiguous grammars and count the number of parses (when possible).

- **Procedure:-**

Step 1: Understand/Define a CFG and convert it to Chomsky Normal Form (CNF). (All productions must be of form $A \rightarrow BC$ or $A \rightarrow a$).

Step 2: Prepare the grammar in a Python-friendly format (dictionaries/lists).

Step 3: Define the input sentence (tokenized).

Step 4: Initialize the CYK table: a triangular 2D structure where $table[i][j]$ holds non-terminals that derive the substring from position i of length j .

Step 5: Fill the table for $length = 1$ using productions $A \rightarrow a$.

Step 6: For lengths $2..n$, fill using combinations of partitions and productions $A \rightarrow B C$.

Step 7: After filling, check if the start symbol (e.g., S) is in $table[0][n]$. If yes — sentence is in the language. Optionally, compute number of parses by storing counts.

Step 8: (Optional) Implement a simple Chart Parser (Earley-like) for general CFGs if needed.

- **Code:-**

```
from collections import defaultdict

class CNFGrammar:

    def __init__(self):

        self productions = defaultdict(list)

        self rhs_to_lhs = defaultdict(list)

    def add_rule(self, lhs: str, rhs):

        """rhs is either a terminal string or a tuple of two nonterminals"""

        self productions[lhs].append(rhs)

        self rhs_to_lhs[rhs].append(lhs)

    def get_lhs_for_rhs(self, rhs):

        return self rhs_to_lhs.get(rhs, [])

def cyk_parse(grammar: CNFGrammar, words, start_symbol="S"):

    n = len(words)

    table = [[set() for _ in range(n + 1)] for _ in range(n)]

    back = dict()

    for i, w in enumerate(words):

        back[(i, 1)] = defaultdict(list)

        lhs_list = grammar.get_lhs_for_rhs(w)

        for A in lhs_list:

            table[i][1].add(A)

            back[(i, 1)][A].append(('terminal', w))
```

Name :- Prathamesh Arvind Jadhav

Roll No:- 4059

```
for l in range(2, n + 1):  
    for i in range(0, n - l + 1):  
        back[(i, l)] = defaultdict(list)  
        for split in range(1, l):  
            left_set = table[i][split]  
            right_set = table[i + split][l - split]  
            for B in left_set:  
                for C in right_set:  
                    lhs_candidates = grammar.get_lhs_for_rhs((B, C))  
                    for A in lhs_candidates:  
                        table[i][l].add(A)  
                        back[(i, l)][A].append(('binary', split, B, C))  
        accepted = start_symbol in table[0][n]  
    return table, back, accepted  
  
def build_trees(back, i, l, symbol):  
    if (i, l) not in back or symbol not in back[(i, l)]:  
        return []  
    trees = []  
    for entry in back[(i, l)][symbol]:  
        if entry[0] == 'terminal':  
            _, word = entry  
            trees.append((symbol, word))
```

Name :- Prathamesh Arvind Jadhav

Roll No:- 4059

```
elif entry[0] == 'binary':  
    _, split, B, C = entry  
  
    left_trees = build_trees(back, i, split, B)  
  
    right_trees = build_trees(back, i + split, 1 - split, C)  
  
    for lt in left_trees:  
  
        for rt in right_trees:  
  
            trees.append((symbol, (lt, rt)))  
  
    return trees  
  
def tree_to_string(tree):  
  
    sym, content = tree  
  
    if isinstance(content, str):  
  
        return f'({sym} {content})'  
  
    left, right = content  
  
    return f'({sym} {tree_to_string(left)} {tree_to_string(right)})'  
  
g = CNFGrammar()  
  
g.add_rule('Det', 'the')  
  
g.add_rule('Det', 'a')  
  
g.add_rule('N', 'dog')  
  
g.add_rule('N', 'cat')  
  
g.add_rule('N', 'park')  
  
g.add_rule('V', 'chased')  
  
g.add_rule('V', 'saw')
```

Name :- Prathamesh Arvind Jadhav

Roll No:- 4059

```
g.add_rule('NP', 'John')
g.add_rule('NP', 'Mary')
g.add_rule('NP', ('Det', 'N'))
g.add_rule('VP', ('V', 'NP'))
g.add_rule('S', ('NP', 'VP'))

sentences = [
    "John saw the dog",
    "the dog chased Mary",
    "Mary saw John",
    "the cat saw the dog in the park"
]

for s in sentences:
    words = s.split()
    table, back, accepted = cyk_parse(g, words, "S")
    print(f"\nSentence: \"{s}\")
    print("Accepted? ", accepted)
    if accepted:
        trees = build_trees(back, 0, len(words), "S")
        print(f"Number of parse trees: {len(trees)}")
        for idx, t in enumerate(trees, 1):
            print(f"Tree {idx}: {tree_to_string(t)}")
    else:
```

```
print("No valid parse tree.")
```

- **Output:-**

```
PS D:\NLP\EXP7> & "C:/Users/H P/AppData/Local/Programs/Python/Python312/python.exe" d:/NLP/EXP7/Exp7.py

Sentence: "John saw the dog"
Accepted? True
Number of parse trees: 1
Tree 1: (S (NP John) (VP (V saw) (NP (Det the) (N dog))))

Sentence: "the dog chased Mary"
Accepted? True
Number of parse trees: 1
Tree 1: (S (NP (Det the) (N dog)) (VP (V chased) (NP Mary)))

Sentence: "Mary saw John"
Accepted? True
Number of parse trees: 1
Tree 1: (S (NP Mary) (VP (V saw) (NP John)))

Sentence: "the cat saw the dog in the park"
Accepted? False
No valid parse tree.
```

- **Question and Answer:-**

Q1. What is CYK parsing and what are its prerequisites (grammar form)?

Ans->

- **CYK Parsing:-**

CYK parsing (Cocke–Younger–Kasami parsing) is a bottom-up dynamic programming algorithm used in natural language processing and formal language theory to determine whether a given string can be generated by a context-free grammar (CFG).

It works by breaking a sentence into substrings and checking systematically which grammar rules can derive those substrings until it builds up to the whole sentence.

- **Prerequisites for CYK Parsing:-**

The grammar must be in **Chomsky Normal Form (CNF)**.

- **CNF Definition:**

A CFG is in CNF if every production rule is of the form:

1. $A \rightarrow BC$ (where A, B, C are non-terminal symbols, B and C are not start symbols)
2. $A \rightarrow a$ (where A is a non-terminal and a is a terminal symbol)
3. Optionally: $S \rightarrow \epsilon$ (only if the language includes the empty string, and only the start symbol can derive ϵ).

Q2. Differentiate between top-down and bottom-up parsing; where does CYK fit?

Ans->

Feature	Top-Down Parsing	Bottom-Up Parsing	CYK
Start Point	Start symbol (root)	Input string (leaves)	Input string
Direction	Derivation (expand)	Reduction (combine)	Reduction
Common Algorithms	LL, recursive descent	LR, shift-reduce	CYK (DP-based)
Grammar Restrictions	Cannot handle left recursion	Handles more grammars	Needs CNF
CYK Belongs To	No	Yes	Yes

Q3. Explain how the CYK table is filled and how the start symbol determines acceptance?

Ans->

1. CYK Table Structure

- The CYK algorithm uses a **triangular table** $T[i][j]$.
- Each cell stores the set of **non-terminals** that can derive a substring of the input.
- Here:
 - i = starting index of the substring (1-based).
 - j = length of the substring.

2. Steps to Fill the Table

Step 1: Initialization (Length = 1 substrings $\rightarrow$ terminals)

- For each symbol of the input string (single characters):
 - If there is a rule $A \rightarrow a$ matching that symbol, put A in the corresponding cell.

Step 2: Filling for Longer Substrings (Length ≥ 2)

- For substrings of length j (2 to n):
 - Split into two parts at every possible position k .
 - If $X \rightarrow YZ$ is a rule and:
 - Y is in the left part cell, and
 - Z is in the right part cell,
 - then add X into the current cell.

Step 3: Acceptance Check

- After filling, look at the **top-right cell** $T[1][n]$ (covers the full string).
- If the **start symbol** S is present $\rightarrow$ string is accepted.
- Otherwise $\rightarrow$ string is rejected.

Q4. Why must the grammar be in CNF for CYK? How do you convert a CFG to CNF?

Ans->

- **CYK Requires CNF:-**

The **CYK algorithm** relies on a simple, structured way of combining rules.

- In **CNF (Chomsky Normal Form)**, rules are only of type:
 1. $A \rightarrow BC$ (two non-terminals)
 2. $A \rightarrow a$ (single terminal)
 3. (optional) $S \rightarrow \epsilon$

This restriction is **essential** because:

1. The CYK table algorithm always checks pairs of cells (left part & right part). This only works if rules are binary ($A \rightarrow BC$).
2. Base cases are simple: single terminals ($A \rightarrow a$) directly map to substrings of length 1.
3. Arbitrary rules like $A \rightarrow BCD$ or $A \rightarrow aB$ would break the **dynamic programming split** idea.

- **How to Convert CFG to CNF**

Converting a **context-free grammar (CFG)** into **CNF** is a systematic process.

Step 1: Eliminate Null (ϵ) Productions

- Remove rules like $A \rightarrow \epsilon$, except if A is the start symbol.
- Adjust other rules to allow optional omission of A.

Step 2: Eliminate Unit Productions

- Remove rules like $A \rightarrow B$ (a single non-terminal).
- Replace them with rules where B leads (e.g., if $B \rightarrow x$, add $A \rightarrow x$).

Step 3: Eliminate Useless Symbols

- Remove non-terminals that:
 - never derive a terminal string, or
 - are never reachable from the start symbol.

Step 4: Convert Productions to Proper CNF Form

- Ensure every production is either $A \rightarrow a$ or $A \rightarrow BC$

- **Conclusion:-**

This experiment demonstrates how CYK uses dynamic programming to fill a parse table and decide whether a string belongs to a CFG in CNF. CYK is efficient ($O(n^3 \cdot |G|)$ time) and well-suited for grammars converted to CNF. Chart parsing (Earley) can parse more general CFGs and can be implemented as an extension.

Name :- Prathamesh Arvind Jadhav

Roll No:- 4059